FC Fluxo de Caixa

DOCUMENTO 12

Desempenho e dados

Índices, paginação keyset, réplica de leitura, cache e evolução do armazenamento

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

1. Índices que já entram 3
2. Paginação keyset 3
3. Particionamento por mês 4
4. Retenção da outbox 4
5. Saldo acumulado 4
6. Réplica de leitura 5
7. Aurora PostgreSQL 6
8. Cache em camadas 7
9. CQRS já é o banco de leitura 7
10. Pool, timeout e diagnóstico 8

FONTE: DOCUMENTOS-BRUTOS/12-DESEMPENHO-E-DADOS.MD

12 — Desempenho e dados

O volume de hoje não pede particionamento, réplica nem outro banco. Este documento fixa o que já vale na primeira versão, o que cada evolução custa, e o número que autoriza a ligá-la. Plano de execução (`EXPLAIN`) das consultas críticas está nos serviços, com o `EXPLAIN (ANALYZE, BUFFERS)` que rodou no banco:

- services/lancamentos-service/docs/performance.md
- services/consolidado-service/docs/performance.md

A decisão abaixo não substitui o plano. O plano é a evidência de que o índice e a janela já bastam neste volume.

Premissas de volume, herdadas do contexto: pico de leitura 50 req/s, pico de escrita 20 lançamentos/s, hit ratio de cache ≥ 80%. Uma escrita média de 1/s (bem abaixo do pico) produz cerca de 31 milhões de linhas por ano. Um comerciante ativo gera, no máximo, uma linha de saldo por dia: 1.800 linhas em cinco anos. Quase toda otimização de leitura de saldo é prematura nesse tamanho. A lista de lançamentos e a outbox são as que podem doer antes.

1. Índices que já entram

Lançamentos. A listagem é `WHERE comerciante_id = ? AND data_lancamento = ? ORDER BY criado_em DESC LIMIT ?` . O índice composto `(comerciante_id, data_lancamento, criado_em DESC)` cobre o filtro e a ordem. Sem a terceira coluna, o plano filtra por comerciante e data e ainda ordena. Com ela, o `LIMIT` para no topo do índice.

O detalhe por id usa a chave primária e **também** `comerciante_id` no predicado. Índice só em `id` sem o predicado de tenant é o bug de BOLA, não uma otimização. Os dois ficam no `WHERE` .

Outbox. Índice parcial `WHERE publicado_em IS NULL` em `(proxima_tentativa_em, id)` . O relay lê dezenas de linhas pendentes, não a história já publicada. Índice na tabela inteira gasta escrita em linha que o `SELECT` do relay nunca mais toca. A coluna `publicado_em` nula é exatamente a fila interna.

Consolidado. A chave primária `(comerciante_id, data)` atende o GET do dia (ponto) e o relatório de período (range em `data` com `comerciante_id` fixo). Não há segundo índice “para o relatório”. `eventos_processados(event_id)` é chave primária: a idempotência é um lookup, não uma varredura.

Índice a mais sem consulta na frente é custo de escrita no caminho do POST. Cada índice novo no lançamentos precisa apontar a query do `EXPLAIN` que ele muda.

2. Paginação keyset

A API de lista é `page` e `size` (offset). Offset 0 com `size` 50, apoiado no índice composto, lê 50 linhas. Offset grande (`page` 10.000, `size` 50) faz o banco contar e descartar 500.000 linhas para devolver 50. Isso não é o uso do caixa diário. É o uso de um extrato que alguém pagina até o fim, ou de um cliente que usa `page` como cursor errado.

Gatilho. p95 da lista acima de 100 ms, ou `EXPLAIN` mostrando linhas removidas por filtro na casa de dezenas de milhares. Abaixo disso, offset fica.

Como, quando o gatilho chegar. Keyset: `WHERE comerciante_id = ? AND data_lancamento = ? AND (criado_em, id) < (:cursorCriadoEm, :cursorId) ORDER BY criado_em DESC, id DESC LIMIT ?`. O cliente manda o cursor da última linha, não um número de página. O índice composto já serve, com `id` no fim para desempate. A resposta ganha `proximaCursor`. A página numerada pode coexistir em `/api/v1` até os clientes migrarem; keyset novo não quebra o contrato velho se for parâmetro opcional.

3. Particionamento por mês

Partição declarativa de `lancamentos` por mês de `data_lancamento` (ou de `criado_em`; os dois só coincidem se a política disser qual é a chave de retenção). O ganho real não é o `SELECT` do dia, que o índice composto já resolve. O ganho é detach de mês antigo e vacuum que não varre a história inteira.

Gatilho, todos ao mesmo tempo não são necessários; basta um.

- A tabela passa de **100 milhões** de linhas (cerca de três anos a 1 escrita/s de média, ou bem antes se o pico de 20/s for sustentado por horas todos os dias).
- Autovacuum não dá conta: idade de transação não congelada subindo, ou o POST espera lock de vacuum no trace.
- A retenção deixar de ser “guardar tudo” e passar a ser “destacar o mês e arquivar no lake”.

Antes disso, partição custa FK, índice único global (a unicidade de `id` precisa incluir a chave de partição, ou o id deixa de ser único sozinho) e Flyway mais difícil, para um `SELECT` que já é indexado. Não se particiona no dia 1.

A outbox **não** se particiona. Ela se limpa.

4. Retenção da outbox

Linha publicada há mais de **7 dias** é apagada por job. Sete dias cobrem uma investigação (“esse evento saiu?”) sem deixar a tabela crescer no ritmo da escrita. A linha não publicada não entra no delete. O índice parcial continua pequeno se o relay estiver saudável. Se não estiver, o alarme de idade (documento 07) dispara aos 30 s, muito antes da retenção importar.

Gatilho para encurtar para 2 dias: a tabela de publicadas, mesmo com o índice parcial, passa a dominar o backup ou o vacuum. **Gatilho para alongar:** um incidente real em que o payload de 7 dias atrás fez falta e o evento no SNS já tinha sido consumido. O fato contábil continua na tabela `lancamento`; a outbox não é a trilha legal.

5. Saldo acumulado

O saldo do dia é uma linha. O saldo acumulado até uma data é a soma dos `saldo` daquele comerciante com `data <= :data`. Para um comerciante, cinco anos são cerca de 1.800 somas de `NUMERIC`. Isso é barato. O relatório de período, se for implementado como 366 consultas correlacionadas, deixa de ser barato por N+1, não pelo volume.

Agora. O período calcula o acumulado em **uma** sentença, com função de janela:

```
SUM(saldo) OVER (PARTITION BY comerciante_id ORDER BY data)
```

no intervalo pedido, mais a soma anterior ao `inicio` buscada uma vez. O `EXPLAIN` dessa sentença é o que o `performance.md` do consolidado deve mostrar. Trezentas e sessenta e seis idas ao banco são defeito, não "simplesidade".

Por que não gravar `saldo_acumulado` na linha, já. Um estorno retroativo muda o saldo de um dia antigo. Todo acumulado posterior daquele comerciante muda junto. Atualizar N linhas futuras na mesma transação do evento acopla a latência do consumidor ao tamanho da história e cria contenção na linha de "hoje" se dois eventos do mesmo comerciante chegarem juntos. A soma é comutativa no dia; o acumulado armazenado **não** é comutativo em relação a um estorno no passado.

Snapshots mensais, quando o gatilho chegar. Guardar, por comerciante e mês fechado, a soma daquele mês. O acumulado até uma data vira "soma dos snapshots dos meses cheios anteriores + soma dos dias do mês corrente". Um estorno em mês fechado recalcula **aquele** snapshot, não os dias um a um. O mês corrente continua sendo a janela.

Gatilho do snapshot. Um destes:

- p99 do relatório de período, com cache frio, acima de **300 ms** por uma semana, com o plano mostrando a janela sobre dezenas de milhares de linhas de um único comerciante (muitos pontos por dia, se o modelo deixar de ser "um saldo por dia");
- ou a história por comerciante acima de **50 mil** linhas de saldo.

Enquanto o modelo for um saldo por dia, 50 mil linhas são mais de um século. O gatilho prático, portanto, é o p99, não a contagem. Se o p99 estourar com 1.800 linhas, o problema é plano ou pool, não falta de snapshot. O `EXPLAIN` distingue os dois.

Materializar o acumulado em coluna, atualizada de forma incremental, só depois do snapshot provar que ainda não basta. Não é o primeiro degrau.

6. Réplica de leitura

Variável `DB_READER_URL`, sempre `jdbc:postgresql://`. Presente: transação `@Transactional(readOnly = true)` vai para o reader, via routing data source e `LazyConnectionDataSourceProxy`. Ausente: tudo vai ao primário, e o código de leitura não muda. O consumidor que faz `UPSERT` **não** é `readOnly`. Mandar a projeção para o reader quebraria o saldo.

O default é zero reader. A AWS não cria **read replica** de RDS for PostgreSQL quando a instância usa senha gerenciada pelo Secrets Manager (limitação). O módulo `rds-postgres` recusa `read_replica_count > 0` com `manage_master_user_password`. O caminho de código fica pronto; o recurso clássico de réplica não sobe nessa combinação.

Três saídas compatíveis, quando o gatilho abaixo disparar:

1. **RDS Multi-AZ DB Cluster** (recomendação). Um escritor e dois standbys legíveis, com reader endpoint. A senha continua gerenciada pelo Secrets Manager: o reader do cluster não é a read replica que a limitação

proíbe. O wrapper usa `failover,awsSecretsManager` no endpoint do cluster. Custa três instâncias, não uma réplica extra de USD 53.

2. **Aurora PostgreSQL** com readers e autoscaling, se a quantidade de readers tiver de variar no dia e o I/O justificar. Mantém senha gerenciada. É o passo da seção 7, não o primeiro.
3. **Senha fora do Secrets Manager**, com rotação por Lambda, e aí a read replica clássica volta a ser aceita pela AWS. Perde a rotação nativa do RDS. Só se as duas opções acima não couberem.

O lançamentos só ganha reader se a lista paginada, e não o POST, for o que satura o primário.

Com RDS Proxy ligado, `DB_URL` aponta para o hostname `*.proxy-*`. O serviço tira o plugin `failover` nesse host: o plugin descobriria a instância e furaria o proxy. O proxy já faz o failover. O wrapper continua só com `awsSecretsManager`.

Gatilho, no consolidado, com evidência de uma semana.

- Credit balance da instância T em queda por causa de **leitura**, com o cache hit ainda acima de 80% (se o hit caiu, o primeiro conserto é o cache, não a réplica); ou
- p99 do GET com cache miss acima de 300 ms e o trace mostrando o tempo no Postgres, não no JWT nem no Redis; ou
- conexões do primário acima de 70% de `max_connections` sustentadas por leituras.

Lag de réplica acima de **5 s** gera alarme e o roteamento de leitura volta ao primário (tirar `DB_READER_URL` ou um disjuntor no routing). Saldo atrasado 5 s na réplica, somado ao atraso da fila, mente o caixa. Prefere-se o primário lento ao número velho sem aviso.

Uma réplica clássica `db.t4g.medium` com 50 GB sairia **USD 53,20/mês** em `us-east-1` (custos), e é exatamente o recurso que a senha gerenciada impede. Não se liga “porque CQRS pede réplica”. O read model já é outro banco. O número de USD 53 fica na tabela como o preço que **não** se paga neste desenho.

RDS Proxy, no mesmo tema de conexão e não de leitura: default desligado. O Advanced JDBC Wrapper já refaz a credencial. Na instância, o plugin `failover` cobre o Multi-AZ. No proxy, esse plugin fica desligado. O Proxy entra quando as conexões de aplicação se aproximam de 70% de `max_connections` (dez tasks do consolidado × pool 20 = 200; um `db.t4g.medium` fica na casa de 450 conexões pela fórmula de memória, então 200 ainda é folga) **ou** quando o tempo de reabrir o pool no failover doer no SLO. Custo de Proxy nos dois bancos: cerca de **USD 44/mês** em `us-east-1`, com a ressalva de preço do documento 08. Ligar Proxy só por rotação de senha é pagar duas vezes pelo que o wrapper já faz.

7. Aurora PostgreSQL

Aurora com storage I/O-Optimized e reader autoscaling é a evolução quando o RDS único deixar de ser o formato certo, não um upgrade estético.

I/O-Optimized vale quando a cobrança de I/O passa de cerca de 25% do compute (a regra prática que a própria AWS usa para esse comparação). No gp3 de hoje, 3.000 IOPS e 125 MB/s já vêm no disco base. O volume deste caixa não chega perto. Enquanto o `EXPLAIN` e o Performance Insights mostrarem IOPS dentro do incluso, Aurora I/O-Optimized é um piso de preço mais alto pelo mesmo trabalho.

Reader autoscaling vale quando o gatilho da réplica da seção 6 dispara **e** a quantidade de readers precisa variar ao longo do dia (fechamento com muita leitura, madrugada com nenhuma). Um reader fixo é mais simples e já está na seção 6. Autoscaling de reader é o passo seguinte, com alarme de lag igual.

Aurora Serverless v2 não escala a zero. O mínimo de 0,5 ACU o mês inteiro, neste volume, empata ou perde de `db.t4g.medium` Multi-AZ e troca o modelo operacional. Fica de fora do default. O documento de custos registra a mesma conclusão.

8. Cache em camadas

Valkey (ElastiCache) é o cache compartilhado entre as tasks do consolidado. Cache-aside do saldo do dia, invalidação por geração do comerciante, TTL de 30 s. Perda do cache degrada para o Postgres. O circuito do Resilience4j torna essa degradação explícita.

Caffeine, no processo, na frente do Redis, guardaria o saldo quente daquela task e o JWKS. A 50 req/s com duas tasks, o Redis vê dezenas de operações por segundo. Isso não aparece no CPU de um `cache.t4g.small`.

Gatilho do Caffeine para saldo. O span do Redis no GET passa de **5 ms** no p95, ou o cache passa de **10 mil** operações por segundo. Abaixo disso, um cache local só cria invalidação duplicada (a geração no Redis já existe para isso) e saldo divergente entre tasks por até o TTL local.

JWKS no Caffeine (ou no cache do Spring) já é outra história: a chave de assinatura muda raramente, e cada request que busca o JWKS no Cognito paga rede. Esse cache é local, com TTL de minutos, e não espera o gatilho acima. Falha de refresh não pode derrubar o POST enquanto a chave em memória ainda for a que assinou o token.

9. CQRS já é o banco de leitura

A projeção em `saldos_diarios` **é** o read model. O write model é o lançamento imutável, noutro Postgres, noutro serviço. Não há um terceiro banco "de leitura" hoje, e não falta: a consulta do caixa é a chave `(comerciante, data)` e uma soma.

Um read model novo só se justifica quando a pergunta deixar de caber nessa chave. Os três candidatos abaixo são alternativas para esse dia, não etapas obrigatórias.

DynamoDB no lugar da projeção

Cabe quando o acesso for ponto (`comercianteId` + `data`) a uma taxa em que as conexões do Postgres, e não a CPU da aplicação, forem o teto — na prática, acima de **1.000 req/s** sustentados de GET, com o cache já na frente. On-demand cobra por request e tira o patch do banco. A um preço de lista da ordem de USD 1,25 por milhão de writes e USD 0,25 por milhão de reads eventualmente consistentes (página pública do DynamoDB em `us-east-1`, ordem de grandeza para a conversa), 50 req/s o mês inteiro são cerca de 130 milhões de reads, dezenas de dólares, **em cima** do RDS que a escrita da projeção ainda exige. Não substitui o RDS até o consumidor também sair do SQL.

O relatório de período e o estorno retroativo são o ponto fraco. DynamoDB não faz a janela `SUM() OVER`. O acumulado vira item materializado, e o estorno antigo reescreve muitos itens. É o problema da seção 5, pior, porque não há transação multi-item barata do tamanho de um ano. DynamoDB ganha o GET do dia em esca-

la alta. Perde o relatório e o estorno. Enquanto os dois existirem no produto, o Postgres da projeção continua o lugar certo.

OpenSearch para relatório analítico

Cabe quando o produto pedir texto livre na descrição, faceta por categoria, ou busca que não é "saldo entre duas datas". Um domínio pequeno ligado o mês inteiro é outra fatura fixa (a menor instância de busca fica na casa de dezenas de dólares por mês, acima do CloudFront desta conta). **Gatilho:** um requisito de busca que, no Postgres, seria `LIKE` ou varredura, com p95 acima do SLO, e que não se resolve com o índice composto. Até lá, OpenSearch é um sistema a operar sem uma pergunta.

Data lake (S3 + Athena) para BI

Cabe quando financeiro ou um analista quiser cruzar meses sem encostar no banco da API. Exportação diária da projeção (e, se a governança permitir, do lançamento sem a descrição, ou com a descrição só no bucket de acesso restrito) em Parquet. Athena cobra da ordem de USD 5 por TB varrido. No volume de um ano deste caixa, a varredura é centavos. O lake não serve o GET da API: latência de segundo e sem o SLO de 100 ms. Serve o relatório que ninguém deve rodar no primário.

Gatilho: uma pergunta recorrente que hoje seria um `SELECT` ad hoc no RDS de produção. A primeira ocorrência já justifica o export. Não justifica trocar a API.

10. Pool, timeout e diagnóstico

Parâmetro	Lançamentos	Consolidado	Motivo
Hikari `maximumPoolSize`	10 por task	20 por task	Teto de conexão. Dez tasks no máximo do scaling dão 100 e 200 conexões, cada uma no seu banco.
`minimumIdle`	2	2	Conexão ociosa ocupa `max_connections`. Dois cobrem o silêncio sem abrir dez na primeira request.
`maxLifetime`	15 min	15 min	Menor que o intervalo de rotação (30 dias em prod) e curto o bastante para soltar conexão velha depois de um failover. O wrapper busca a senha nova; o pool não fica com socket de horas.
`connectionTimeout`	2 s	2 s	Falhar rápido quando o banco não aceita mais conexão, em vez de segurar a thread virtual à espera.
`statement_timeout` do usuário da API	2 s	2 s	Uma sentença ruim não segura conexão por dezenas de segundos.
`statement_timeout` do usuário do consumidor	—	5 s	O `UPSERT` é curto; 5 s separam "lento" de "travado" sem matar um plano legítimo no catch-up.

Usuários de banco separados (API, consumidor, migração Flyway) existem para o timeout e o grant não serem globais. Flyway não usa o usuário da API.

`pg_stat_statements` ligado, para achar a sentença que o `EXPLAIN` pontual não mostra de novo. Performance Insights com retenção de 7 dias, que fica na franquia, para ver espera (IO, lock, CPU) no mesmo intervalo do alarme. Os dois são o laço: Insights diz "está esperando lock"; `pg_stat_statements` diz qual query;

o `performance.md` guarda o plano.

O que não se faz: log de SQL em INFO em produção, e métrica de latência com `comerciante_id` como label. Os dois vazam ou explodem cardinalidade, e nenhum substitui o `EXPLAIN`.